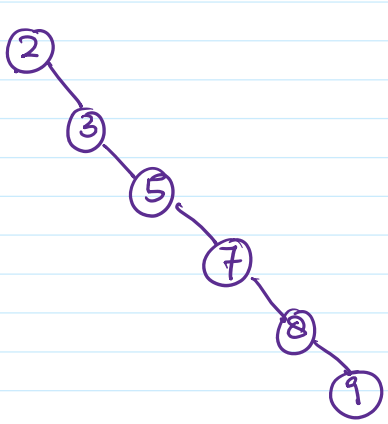
Lecture 8 Binary Search Tree

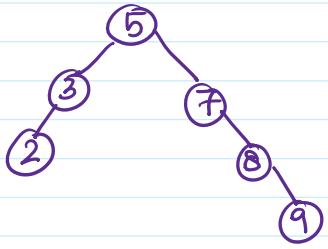
11 August 2026 11:10

A binary search tree is a binary tree s.t.

- Each internal node v stores (k, val) of a dictionary
- keys stored in left subtree of v are less than k
- keys stored in right subtree of v are greater than k

Ex: 2, 3, 5, 7, 8, 9

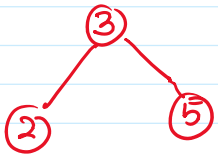


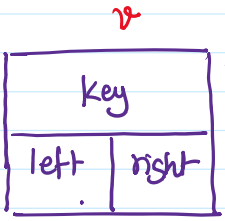


height of

Let h be ↑ BST consisting of n nodes

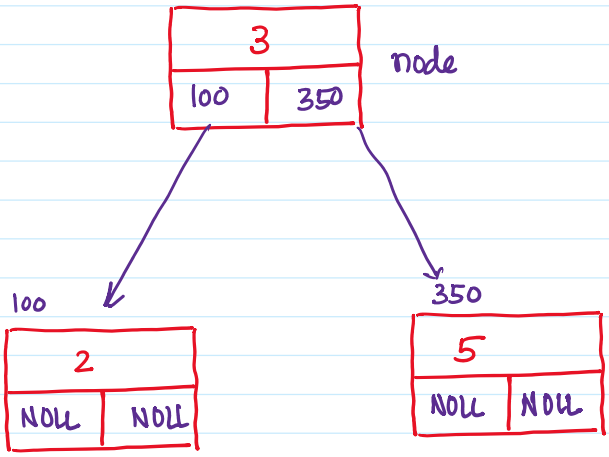
$$\log n \leq h \leq n-1$$





left := ptr to left subtree of v

right := ptr to right subtree of v



node → key := output key value stored in node

node → left := output ptr to left subtree of node

Search in BST

Input: root := ptr to root node of BST

Search in BST

Input: root := ptr to root node of BST

key := given key

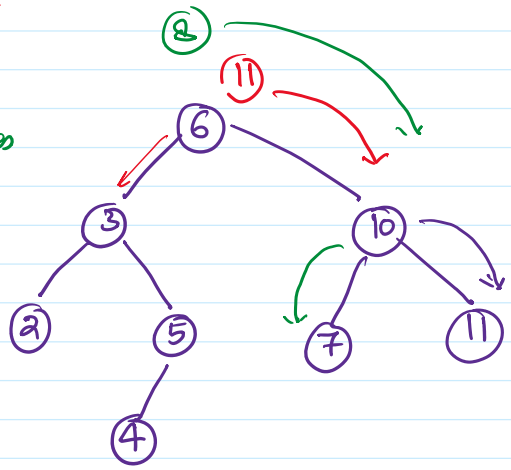
Output: Yes/No depending key is present in BST

Search 11 - Yes

Search 8 - NO

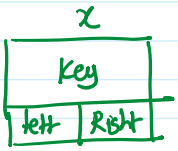
Time Complexity for Search O(h)

h := height of BST



Finding Minimum of BST

```
Tree minimum (root)
    x ← root
    while (x→left ≠ NULL)
        x ← x→left
    return x
```



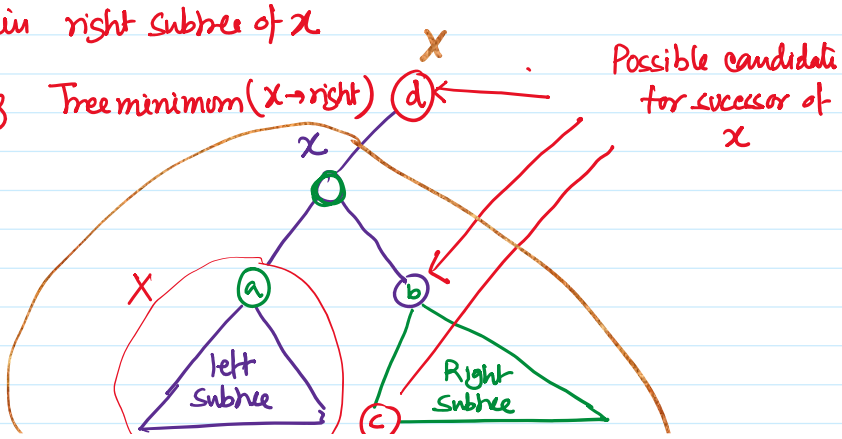
Time Complexity O(h)

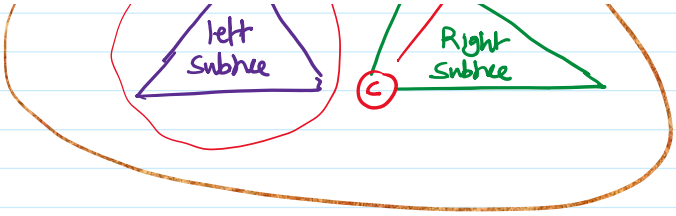
Finding Successor of a given node

Given a node x find node with the smallest key greater than (x→key)

Case 1 Right subtree of x is non-empty

- Successor is leftmost node in right subtree of x
- Can be computed by calling Tree minimum(x→right)





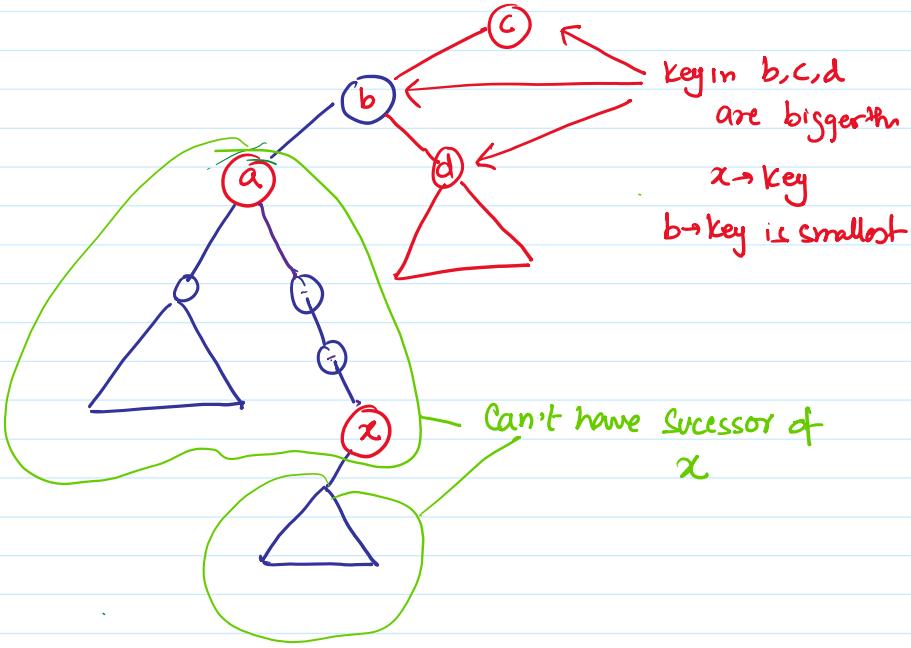
Case 2 Right subtree of x is empty

$b \to key < c \to key$

$d \to key > b \to key$

$\Rightarrow$ $b \to key$ is smallest

$\Rightarrow$ Sucessor of x is lowest ancestor of x whose left child is also ancestor of x




```
Tree Sucessor ( x )
    if (x → right ≠ NULL)
        [ return Tree minimum (x → right)

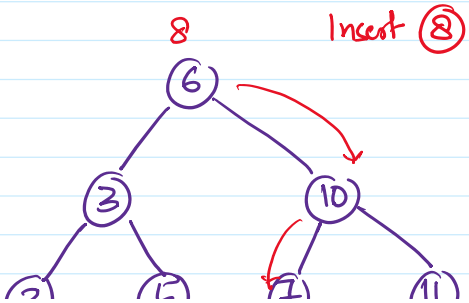
    y := x → parent
    while (y ≠ NULL and x == y → right)
            [ x ← y
              y ← y → parent
        return y
```

Time complexity O(h)

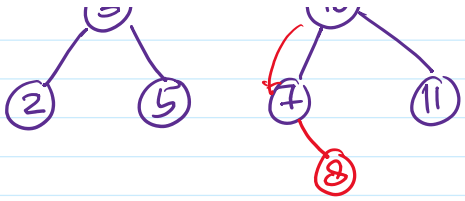
BST Insertion

Input: [A BST - ptr to BST
       [key

Time Complexity O(h)



Time Complexity O(h)



Deletion in BST

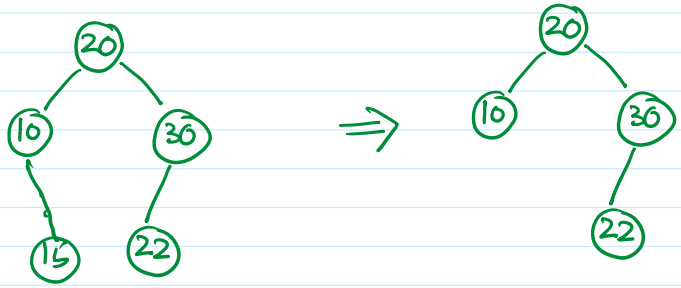
Input $\begin{cases} T & \text{BST} - \text{ptr to root} \\ \text{Key} := x \end{cases}$

Output BST := $T \setminus \{Key\}$

Input BST T and key K

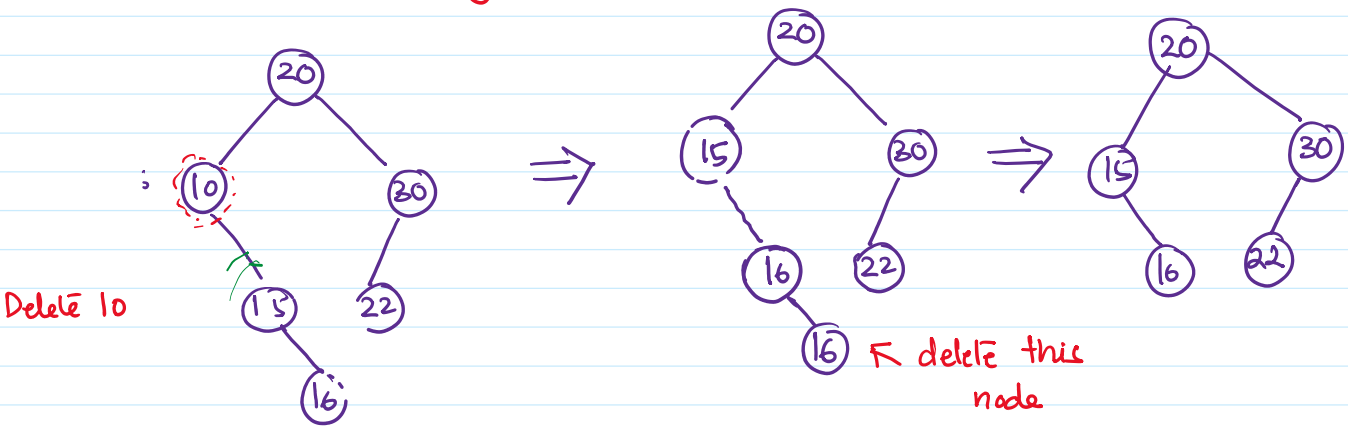
Output BST $T \setminus \{K\}$

Case 1 node x is leaf node



Delete 15

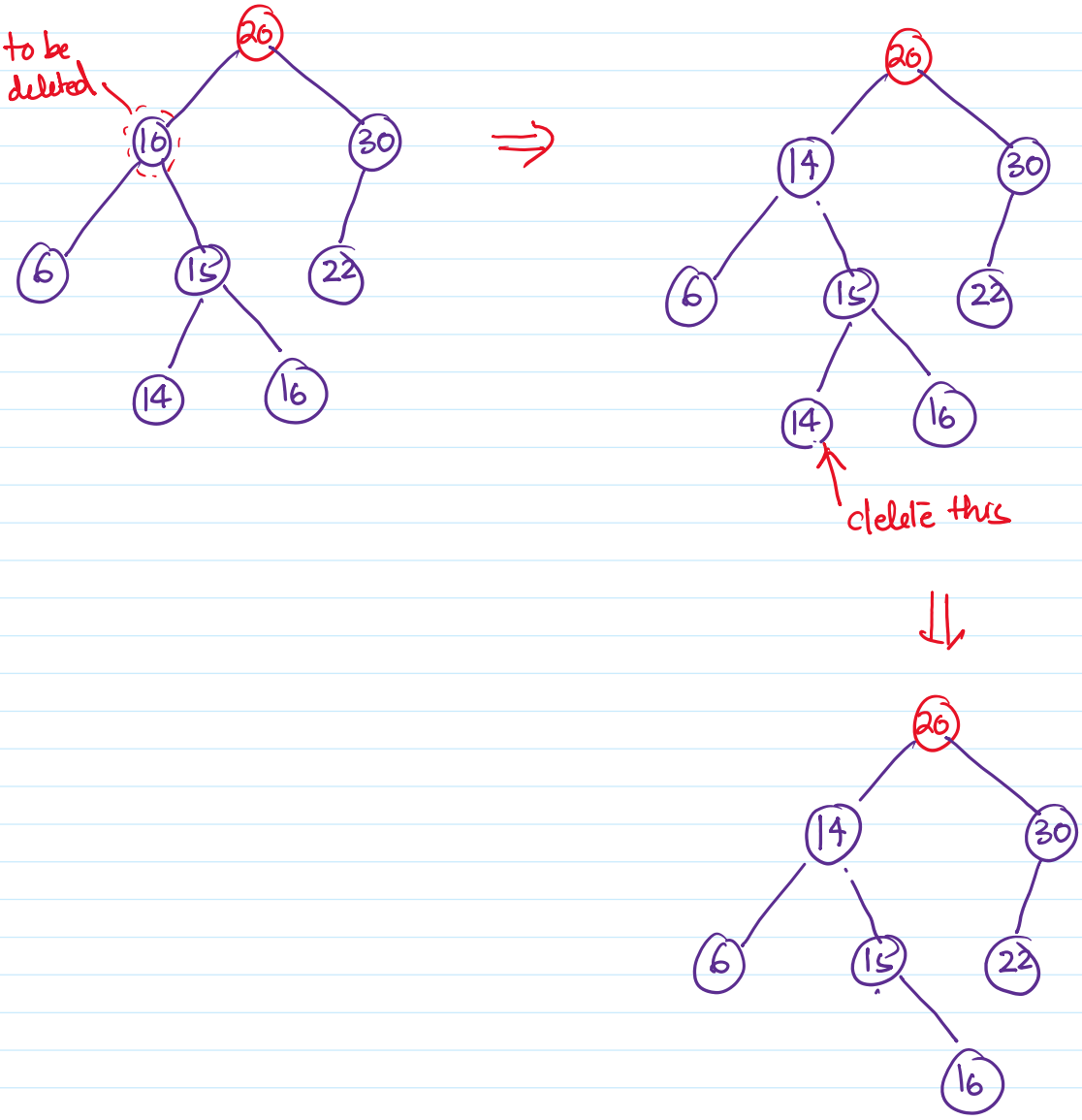
Case 2 Node x has only one child



Delete 10

← delete this node

Case 3 Node x has both left and right child


to be deleted
20
16
30
6
15
22
14
16
20
14
30
6
15
22
14
16
delete this
20
14
30
6
15
22
16